

The next three lines of code create a Wi-Fi access point (AP) node, which allows wireless devices to connect to a wired network. Since we also have a wired section in this network, a Wi-Fi AP node is mandatory. This Wi-Fi AP node is also added to the common node container object *allNodes*, as follows.

```
NodeContainer wifiApNode;
wifiApNode.Create(1);
allNodes.Add(wifiApNode);
```

The following lines of code define the channel used by the Wi-Fi nodes. The *YansWifiChannelHelper* class creates a channel object which implements the YANS channel model. The YANS channel implements the propagation model described in ‘Yet Another Network Simulator’, a paper published by Mathieu Lacage and Thomas R. Henderson. You can get the paper at <http://cutebugs.net/files/wms2-yans.pdf>. The default values of the YANS channel are used in this simulation.

```
YansWifiChannelHelper channel = YansWifiChannelHelper::Default
();
YansWifiPhyHelper phy = YansWifiPhyHelper::Default ();
```

The statement *phy.SetChannel(channel.Create ());* in the script makes sure that all the physical layer objects share the same underlying channel.

The next few lines of code configure the MAC layer. The class *Ssid* is responsible for setting up the SSID (service set identifier). SSID is a unique identifier attached to the packet header in wireless communication. All the nodes which are connected to a particular network should contain the same SSID.

```
WifiHelper wifi = WifiHelper::Default ();
wifi.SetRemoteStationManager("ns3::AarfwifiManager");
NqosWifiMacHelper mac = NqosWifiMacHelper::Default ();
Ssid ssid = Ssid("ns-3-ssid");
mac.SetType("ns3::StaWifiMac",
    "Ssid", SsidValue(ssid),
    "ActiveProbing", BooleanValue(false));
```

The next two lines of code use the *Install()* method to create Wi-Fi devices on the two Wi-Fi station nodes. But we haven't dealt with any code configuring the Wi-Fi AP node. But in the downloaded full script of *wireless.cc*, you will see code to handle the MAC layer of the Wi-Fi AP node.

```
NetDeviceContainer staDevices;
staDevices = wifi.Install(phy, mac, wifiStationNodes);
```

After handling the physical and MAC layers of the Wi-Fi nodes, we also have to create a node connected

to the Wi-Fi AP node by a point-to-point link (wired connection). Since we are familiar with this procedure, I haven't explained it. The line *p2pNodes.Add(wifiApNode);* connects the point-to-point node object *p2pNodes* with the Wi-Fi AP node object *wifiApNode*. So this line of code connects the wired and wireless portions of our simulated network. After this, an object of the *MobilityHelper* class is created to fix the position and mobility model of various types of nodes. A method of class *MobilityHelper* called *SetPositionAllocator()* is used to fix the initial position of nodes before starting the simulation. Another method called *SetMobilityModel()* is called to fix the mobility model associated with a node. In our simulation, the two Wi-Fi nodes are assigned with a *RandomWalk2dMobilityModel*. This makes the movement of Wi-Fi nodes random. So the movement of the two Wi-Fi station nodes change with each execution of the script. The Wi-Fi AP node is assigned with a *ConstantPositionMobilityModel* and hence the Wi-Fi AP node is stationary. Finally the *Install()* method of *MobilityHelper* class is used to install the various mobility models on different nodes.

After this, the script *wireless.cc* proceeds like the previous scripts we have analysed. Just like previous simulation scripts, IPv4 addresses are allocated to all the nodes. An application is installed on top of the simulated nodes. A port address of 5555 is used by the server application to communicate with the client application.

Now, only the following two lines of code need explanation. These lines deal with the animation trace file. In the code *anim* is an object of the class *AnimationInterface*. Here, the first line of code accesses one Wi-Fi station node and labels it as *Wireless_N1*. The second line of code sets the colour of the Wi-Fi station node in the *NetAnim* animation. An ordered triplet corresponding to the RGB colour code is used to denote the colour of the nodes. Here, we have the triplet as (255, 0, 0) and you can see a red coloured Wi-Fi station node in the *NetAnim* window.



Figure 1: Wireless nodes in NetAnim